ИТОГОВЫЙ ОТЧЁТ

Архитектура и организация разработки

Кейс «Новостная аналитика для принятия бизнес-решений»

Рекомендуемое решение

Единый Git-репозиторий с модулями в отдельных папках, внутренние вызовы через Python-функции и один FastAPI для взаимодействия с frontend. Это снижает интеграционные риски и подходит команде из трёх человек при сроке 3–7 дней.

Команда: Python / Data • LLM • Full-stack + Team Lead

1. Резюме и решение

Рекомендуется не выбирать между модулями в одном проекте и отдельным API для каждого модуля как взаимоисключающими вариантами. Оптимальный вариант — модульный монолит: единый проект, отдельные модули и один внешний API-слой.

Ключевой принцип

API не заменяет модули, а предоставляет к ним единый вход. Внутри приложения модули вызываются обычными Python-функциями; frontend взаимодействует с backend через HTTP API.

Почему это подходит команде

- Участник 1 — Python / Data Engineer; участник 2 — LLM Engineer; участник 3 — Full-stack Engineer и вероятный Team Lead.
- Срок ограничен 3–7 днями, поэтому важны быстрый запуск, простая отладка и ранняя интеграция.
- Единый репозиторий облегчает совместную разработку, проверку кода и итоговую сдачу.
- Отдельные сервисы добавят сетевые вызовы, конфигурацию, тайм-ауты, обработку ошибок и задачи развёртывания.
- Модули можно разрабатывать параллельно, если заранее согласовать интерфейсы и JSON-схемы.

2. Предлагаемая архитектура

Поток запроса: пользователь выбирает компанию → frontend вызывает FastAPI → сервисный слой получает профиль и релевантные события → AI-модуль формирует и проверяет инсайты → API возвращает структурированный ответ → frontend отображает результат и источники.

FRONTEND — участник 3

Выбор компании, новости, карточки инсайтов, переход к источникам.

↓ HTTP

Внешний вызов через единый API.

FASTAPI — единая точка входа

Маршруты, валидация запросов, обработка ошибок и передача управления сервисному слою.

↓ внутренние вызовы

Python-функции; без отдельного сетевого API между модулями на старте.

МОДУЛЬ ДАННЫХ — участник 1

Профиль компании, обработка новостей, поиск материалов, ранжирование.

AI-МОДУЛЬ — участник 2

LLM-клиент, агентный workflow, генерация инсайтов, самопроверка и источники.

Сервисный слой app/services/insight_service.py координирует последовательность вызовов. Он не должен дублировать логику обработки данных или генерации AI-инсайтов.

3. Рекомендуемая структура репозитория

```
news-analytics/
├── app/
│   ├── main.py
│   ├── api/
│   │   ├── routes_companies.py
│   │   └── routes_insights.py
│   ├── services/
│   │   └── insight_service.py
│   ├── data/
│   │   ├── company_profile.py
│   │   ├── news_parser.py
│   │   ├── ranking.py
│   │   └── retrieval.py
```

```
│   ├── ai/
│   │   ├── agent.py
│   │   ├── llm_client.py
│   │   ├── prompts.py
│   │   └── validators.py
│   ├── schemas/
│   │   ├── company.py
│   │   └── insight.py
│   └── config.py
├── frontend/
├── data/
│   ├── raw/
│   └── processed/
├── tests/
├── Dockerfile
├── compose.yaml
├── requirements.txt
├── .env.example
├── .gitignore
├── README.md
└── THIRD_PARTY_NOTICES
```

Назначение ключевых директорий:

Путь	Ответственный	Назначение
app/data/	Участник 1	Профили, новости, ранжирование и retrieval
app/ai/	Участник 2	LLM, агентная логика, промпты и валидация
app/api/	Участник 3	HTTP-endpoints
frontend/	Участник 3	Пользовательский интерфейс
app/services/	Участник 3 координирует	Сценарий, соединяющий модули
app/schemas/	Совместно	Единые форматы входных и выходных данных

4. Как вызываются модули

Внутренние модули вызываются функциями. Сервисный слой получает профиль, выбирает релевантные события и передаёт их в AI-модуль:

```
# app/services/insight_service.py

from app.data.company_profile import get_company_profile
from app.data.ranking import get_relevant_news
from app.ai.agent import generate_insight

def build_company_insights(company_id: str):
    profile = get_company_profile(company_id)
    news = get_relevant_news(profile)
    insights = []
    for event in news:
        insights.append(generate_insight(
            company_profile=profile, event=event
        ))
    return {"company_id": company_id, "insights": insights}
```

Это иллюстрация границ модулей, а не готовый production-код. Реальные сигнатуры и схемы необходимо согласовать команде.

Внешний API для frontend

```
from fastapi import APIRouter
from app.services.insight_service import build_company_insights

router = APIRouter(prefix="/api")
```

```
@router.get("/companies/{company_id}/insights")
def get_insights(company_id: str):
    return build_company_insights(company_id)
```

- Frontend знает только API и формат ответа, но не импортирует Python-модули напрямую.
- API вызывает сервисный слой; сервисный слой координирует модули данных и AI.
- Модули данных и AI не должны зависеть от frontend.
- Если анализ длится дольше допустимого времени HTTP-ответа, предусмотреть POST для запуска задачи и отдельный endpoint получения результата. Для MVP можно начать с синхронного вызова, если он достаточно быстрый.

5. Почему не отдельный API для каждого модуля

Критерий	Модульный монолит	Отдельные API-сервисы
Скорость старта	Высокая	Ниже: больше настройки
Отладка	Проще, вызовы внутри процесса	Сложнее: сетевые ошибки и логи
Развёртывание	Один backend и frontend	Несколько сервисов и конфигураций
Независимое масштабирование	Ограничено	Удобнее
Подходит сроку 3–7 дней	Да, рекомендуется	Обычно избыточно без готовой инфраструктуры

Отдельные API-сервисы стоит рассматривать, если компоненты нужно независимо масштабировать или развёртывать, уже существует инфраструктура сервисов либо есть явные требования к изоляции. Таких условий в текущем описании не установлено.

6. Git и параллельная разработка

- Один общий репозиторий для всего решения.
- Участник 1 работает в ветке feature/company-data.
- Участник 2 работает в ветке feature/llm-agent.
- Участник 3 работает в ветке feature/api-frontend и координирует интеграцию.
- Использовать pull request и небольшие изменения; не держать большие незавершённые ветки до конца хакатона.
- Сначала согласовать сигнатуры функций и JSON-схемы, затем параллельно разрабатывать модули.
- Сохранять историю коммитов и зафиксировать идентификатор версии для сдачи.

7. План действий на первый день

1. Создать общий Git-репозиторий и структуру папок.
2. Согласовать модели CompanyProfile, NewsEvent и Insight.
3. Участнику 1 определить сигнатуры функций профиля и поиска новостей.
4. Участнику 2 определить сигнатуру генерации инсайта и формат источников.
5. Участнику 3 поднять FastAPI и простой frontend с тестовыми данными.
6. Проверить полный сценарий на заглушках до подключения реальной LLM.

8. Важные технические ограничения

- Использовать только разрешённые кейсом модели LLM и самостоятельно реализовать агентный workflow; не использовать внешние AI-сервисы или готовые мультиагентные системы как основной компонент.
- Не усложнять архитектуру количеством агентов: число агентов само по себе не является критерием успеха.
- Проверить доступность разрешённой модели до построения зависимых от неё компонентов.
- Контролировать число вызовов LLM и лимит бюджета; кэшировать результаты там, где это корректно.
- Не подменять отсутствующие факты выдуманными; явно показывать пробелы в данных и ограничения.
- Сохранять связь ключевых выводов с первоисточниками и разделять факты, позицию банка и выводы AI.
- Подготовить Docker, README, lock-файлы, THIRD_PARTY_NOTICES и синтетические тестовые данные.

9. Риски и способы их снизить

Риск	Меры снижения
Модули не совместимы	Зафиксировать сигнатуры и JSON-схемы в первый день; использовать тестовые заглушки.
Слишком поздняя интеграция	Показать первый инсайт в интерфейсе не позднее второго дня.
LLM недоступна или дорогая	Проверить доступ заранее, ограничить вызовы, обработать ошибки и тайм-ауты.
Инсайт не обоснован источниками	Требовать источники в схеме результата; валидировать и показывать ограничения.
Участник 3 перегружен координацией	Короткие ежедневные синки, самостоятельная ответственность владельцев модулей и чёткие контрольные точки.
Нет времени на упаковку	Рано добавить Docker и README; оставить время на чистый запуск и фиксацию версии.

10. Итоговая рекомендация

Выбор

Единый репозиторий + отдельные папки модулей + Python-функции для внутренних вызовов + один FastAPI для frontend.

Этот подход позволяет разрабатывать компоненты параллельно, не создавать лишнюю инфраструктуру и оставить возможность позже вынести отдельные модули в сервисы, если появится реальная необходимость.

Ключевой критерий готовности — не число сервисов и не сложность архитектуры, а полный воспроизводимый сценарий: выбор компании, получение профиля, отбор события, персонализированная интерпретация с источниками и отображение результата в интерфейсе.

Основание

Отчёт оформлен по последнему обсуждению архитектуры и с учётом условий кейса «Новостная аналитика для принятия бизнес-решений». Примеры кода и структуры — рекомендуемый вариант реализации, который команде следует адаптировать к фактическому стеку и доступным данным.